

Livello Applicazione – Risposte Dettagliate

Sezione 8.1 – Posta Elettronica: MIME, SMTP, IMAP, POP3

1. Quali sono i due sottosistemi che compongono l'architettura di un sistema di posta elettronica?

I due sottosistemi sono:

- **User Agent (agenti utente)** : programmi che consentono alle persone di leggere, comporre, rispondere, visualizzare e organizzare i messaggi di posta.
- **Message Transfer Agent (agenti di trasferimento dei messaggi, MTA)** : processi di sistema (mail server) che spostano i messaggi dalla sorgente alla destinazione.

2. Cosa sono gli user agent (agenti utente)? Quali funzioni svolgono?

Sono programmi che forniscono un'interfaccia (grafica, testuale o basata su comandi) per interagire con il sistema di posta elettronica. Le funzioni includono: comporre, rispondere, visualizzare, organizzare i messaggi (filtrarli, cercarli, scartarli). Alcuni user agent offrono funzioni avanzate come la compilazione automatica di risposte o il filtraggio dello spam.

3. Cosa sono i message transfer agent (MTA)? Dove vengono eseguiti e a cosa servono?

Sono tipicamente processi di sistema eseguiti come servizi sui mail server, pensati per essere sempre disponibili. Il loro compito è spostare automaticamente le e-mail attraverso il sistema dalla sorgente al destinatario facendo uso di **SMTP (Simple Mail Transfer Protocol)** .

4. Quale protocollo utilizzano i message transfer agent per spostare le e-mail?

Utilizzano **SMTP (Simple Mail Transfer Protocol)** , specificato nell'RFC 5321 (originariamente RFC 821). SMTP invia le e-mail con un approccio orientato alla connessione e notifica lo stato della consegna e gli errori.

5. Cosa sono le mailbox (caselle di posta)? Chi le mantiene?

Sono caselle che memorizzano le e-mail ricevute dall'utente. Sono mantenute dai **mail server** (MTA). Gli user agent presentano agli utenti una visione dei contenuti delle loro mailbox, inviando comandi per manipolarle (ispezionare contenuti, cancellare messaggi, etc.).

6. Qual è la differenza tra involucro (envelope) e contenuto di un messaggio di posta elettronica?

- **Involucro**: contiene tutte le informazioni necessarie per il trasporto (indirizzo di destinazione, priorità, livello di protezione). I message transfer agent utilizzano l'involucro per il routing.
- **Contenuto**: consiste di intestazione (informazioni di controllo per gli user agent) e corpo (dedicato al destinatario umano).

7. Quali sono le due parti che compongono il messaggio all'interno dell'involucro?

L'intestazione (header) e il corpo (body). L'intestazione contiene informazioni di controllo (es. To, From, Subject), mentre il corpo è il contenuto effettivo del messaggio destinato all'utente.

Sottosezione 8.1.1 – Formato dei Messaggi

8. Quali sono le tre componenti principali di un messaggio di posta elettronica secondo l'RFC 5321?

Secondo l'RFC 5321 (che descrive SMTP), le tre componenti principali sono:

1. **Involucro** (envelope)
2. **Campi di intestazione** (header fields)
3. **Corpo del messaggio** (body)

9. A cosa serve il campo "To" nell'intestazione di un messaggio?

Indica gli indirizzi DNS dei destinatari **principali** del messaggio (destinatari diretti).

10. Qual è la differenza tra i campi "Cc" e "Bcc"?

- **Cc (Carbon Copy)** : elenca gli indirizzi dei destinatari **secondari** (copie per conoscenza). Questi indirizzi sono visibili a tutti i destinatari.
- **Bcc (Blind Carbon Copy)** : simile a Cc, ma **nascosto** ai destinatari principali e secondari. I destinatari elencati in Bcc non sono visibili agli altri.

11. Qual è la differenza tra i campi "From" e "Sender"?

- **From**: identifica chi ha **scritto** il messaggio (l'autore).
- **Sender**: indica chi ha **effettivamente inviato** il messaggio (può essere diverso dal mittente, ad esempio un segretario che invia per conto del capo).

12. A cosa serve il campo "Received" aggiunto da ogni MTA?

È un campo aggiunto da ogni MTA (mail server) lungo il percorso del messaggio. Contiene informazioni sulla ricezione del messaggio (timestamp, identità del server, etc.). Serve per il tracciamento del percorso e per il debugging.

13. Cosa indica il campo "Return-Path"?

Fornisce un modo per **ricontattare il mittente**, tipicamente utilizzato per i messaggi di errore (es. notifiche di mancata consegna). Indica l'indirizzo a cui inviare le notifiche di bounce.

14. A cosa serve il campo "Message-Id"?

È un numero univoco (generato dal sistema) per identificare il messaggio in modo univoco a livello globale. Serve a prevenire consegne duplicate e a tracciare i messaggi.

15. A cosa serve il campo "Reply-To"?

Specifica a chi **rispondere** (indirizzo di risposta). È utile quando il mittente e l'inviatore sono diversi: ad esempio, un messaggio inviato da un segretario (Sender) ma si vuole che le risposte vadano al capo (Reply-To).

To).

16. Cosa sono le intestazioni personalizzate che iniziano con "X"?

Sono intestazioni create dagli utenti per aggiungere informazioni specifiche o personalizzate non previste dagli standard. L'RFC 5321 riconosce la possibilità di crearle. Esempi: X-Priority, X-Mailer, X-Spam-Status.

Sottosezione 8.1.1.1 – RFC 5322 (ex RFC 822)

17. Quale RFC descrive il formato originale dei messaggi di posta elettronica ASCII?

L'**RFC 822** (e la sua ultima revisione **RFC 5322**). Descrive la struttura dei messaggi di posta elettronica con intestazioni in formato ASCII.

Sottosezione 8.1.1.2 – MIME

18. Perché è stato sviluppato MIME (Multipurpose Internet Mail Extension)?

Per **superare le limitazioni** dei messaggi di posta elettronica basati esclusivamente su testo ASCII. MIME consente l'invio di contenuti più complessi, come lingue con accenti, alfabeti non latini (cirillico, arabo, etc.), ideogrammi (cinese, giapponese) e file non testuali come immagini, audio, video e documenti binari.

19. Quali limitazioni dei messaggi ASCII supera MIME?

- Testo in lingue che non usano l'alfabeto latino (accenti, caratteri speciali)
- Alfabeti non latini (cirillico, arabo, greco, etc.)
- Ideogrammi (cinese, giapponese, coreano)
- File binari (immagini, audio, video, documenti eseguibili, PDF, etc.)
- Messaggi con più parti (allegati)

20. Quali sono le cinque nuove intestazioni introdotte da MIME?

1. **MIME-Version**: indica la versione MIME utilizzata
2. **Content-Description**: descrive il contenuto del messaggio (aiuta il destinatario a decidere se leggerlo)
3. **Content-Id**: identifica univocamente il contenuto (simile a Message-Id)
4. **Content-Transfer-Encoding**: specifica la codifica utilizzata per trasmettere il contenuto via SMTP
5. **Content-Type**: indica la natura del corpo del messaggio (tipo di dati)

21. A cosa serve il campo "Content-Type" in MIME?

Indica la **natura del corpo del messaggio** e il tipo di dati, come testo, immagini, audio, video, applicazioni, etc. Viene specificato come "tipo/sottotipo", ad esempio text/plain, image/jpeg, application/pdf.

22. A cosa serve il campo "Content-Transfer-Encoding"? Quali schemi di codifica esistono?

Specifica la **codifica utilizzata per trasmettere il contenuto tramite SMTP** (che originariamente era progettato solo per ASCII). I cinque schemi di codifica comuni sono:

- **ASCII** (7 bit)

- **8 bit**
- **Binario** (binary)
- **Base64** (per dati binari)
- **Quoted-printable** (per testi con caratteri speciali)

23. Elenca almeno cinque tipi MIME con relativi sottotipi (es. text/plain).

- **text/plain** – testo semplice
- **text/html** – pagina web HTML
- **image/jpeg** – immagine JPEG
- **image/png** – immagine PNG
- **audio/mpeg** – audio MP3
- **video/mp4** – video MP4
- **application/pdf** – documento PDF
- **application/zip** – archivio ZIP
- **application/json** – dati JSON
- **multipart/mixed** – messaggio con allegati

24. Cosa permettono di fare i tipi multipart (multipart/mixed, multipart/alternative, multipart/related)?

- **multipart/mixed**: permette di comporre messaggi con allegati (parti separate)
- **multipart/alternative**: permette di fornire lo stesso contenuto in formati alternativi (es. text/plain e text/html) – il client sceglie il formato più adatto
- **multipart/related**: permette di includere risorse correlate (es. immagini inline in un messaggio HTML)

25. Quali sfide legate alla sicurezza presenta l'uso di MIME?

Quando si eseguono o interpretano contenuti ricevuti (es. allegati eseguibili, script, macro) ci sono **sfide legate alla sicurezza**. Un allegato malevolo potrebbe essere eseguito automaticamente dall'user agent se non gestito correttamente, causando infezioni o violazioni della sicurezza. È fondamentale validare e sanitizzare i contenuti ricevuti.

Sottosezione 8.1.2 – Consegna Finale

26. Perché SMTP non è adatto per la consegna finale della posta al client? (cos'è un protocollo push?)

SMTP è un protocollo **push** (di spinta): prende un messaggio e si connette al server remoto per trasferirlo. Non è adatto per la consegna finale perché:

- La casella di posta deve continuare a essere memorizzata sul mail server (non scaricata e cancellata)
- Lo user agent potrebbe non essere connesso a Internet nel momento in cui SMTP tenta di inoltrare i messaggi
- Serve un protocollo **pull** (di prelievo) che permetta al client di recuperare i messaggi quando è connesso

27. Quali sono i protocolli utilizzati per la consegna finale della posta?

I principali protocolli sono **IMAP** (Internet Message Access Protocol) e **POP3** (Post Office Protocol versione 3), oltre a protocolli proprietari (es. Microsoft Exchange) e Webmail (che usa HTTP/HTTPS).

Sottosezione 8.1.2.1 – IMAP

28. Cosa significa l'acronimo IMAP? Su quale porta ascolta il server IMAP?

IMAP significa **Internet Message Access Protocol** (protocollo di accesso ai messaggi Internet). La sua versione più recente è definita nell'RFC 3501. Il server IMAP ascolta sulla **porta 143**.

29. Quali operazioni può eseguire un client IMAP dopo l'autenticazione?

Può eseguire numerose operazioni, tra cui:

- Elencare le cartelle (mailbox)
- Visualizzare i messaggi
- Etichettare i messaggi
- Spostare i messaggi tra le cartelle
- Cercare messaggi per vari criteri
- Gestire le flag (letto, risposto, cancellato, etc.)

30. Quali sono i principali miglioramenti di IMAP rispetto a POP3?

- **IMAP mantiene i messaggi sul server**, mentre POP3 di solito li scarica sul client (e li cancella dal server)
- IMAP permette la **sincronizzazione tra diversi dispositivi** (stato dei messaggi coerente su PC, telefono, tablet)
- IMAP permette di **gestire le cartelle** sul server
- Con POP3, se il client si guasta, i messaggi possono essere persi
- IMAP supporta accesso **parziale** ai messaggi (si possono scaricare solo gli header prima di decidere)

31. Quale RFC definisce POP3?

POP3 è definito nell'**RFC 1939**.

Sottosezione 8.1.2.2 – Webmail

32. Come funziona il Webmail? Quale protocollo usa il browser per comunicare con il server?

Il provider (es. Gmail, Hotmail) gestisce un server di posta che accetta messaggi via SMTP (porta 25). Lo user agent è un'interfaccia web accessibile tramite **browser**. L'utente accede alla pagina web del servizio, inserisce le credenziali (inviata al server via HTTP/HTTPS per la convalida). Se autenticato, il server recupera la casella di posta e genera una pagina web che viene inviata al browser (via **HTTP** o **HTTPS**). Per inviare un nuovo messaggio, l'utente compila un modulo web e invia i dati al server sempre con HTTP.

33. Quali sono i principali provider di Webmail menzionati nel testo?

- **Google Gmail**
- **Microsoft Hotmail**

- **Yahoo! Mail**

34. Come avviene l'invio di un nuovo messaggio in un sistema di Webmail?

L'utente compila un modulo web (con destinatario, oggetto, corpo del messaggio, eventuali allegati). I dati vengono inviati al server utilizzando il normale protocollo **HTTP** (o **HTTPS** per la sicurezza). Il server Web si occupa quindi di inviare il messaggio attraverso il sistema di consegna tradizionale, utilizzando i protocolli standard della posta (SMTP verso i server di destinazione).

Sezione 8.2 – Protocolli di Trasporto File

35. Quali sono i due protocolli di trasporto file descritti nel capitolo?

FTP (File Transfer Protocol) e **TFTP (Trivial File Transfer Protocol)**. Differiscono significativamente nelle funzionalità e nel modo in cui operano.

Sottosezione 8.2.1 – FTP

36. Cosa significa l'acronimo FTP? A cosa serve?

FTP significa **File Transfer Protocol** (protocollo di trasferimento file). È stato progettato per consentire agli utenti di trasferire file in modo efficiente e affidabile da un host all'altro attraverso Internet, offrendo la possibilità di replicare dati per backup su larga scala.

37. Quali due tipi di accesso offre FTP?

- **Accesso autenticato:** richiede una coppia di account/password per l'autenticazione dell'utente
- **Accesso anonimo:** di solito senza restrizioni (si accede con "anonymous" come username e email come password, tipico per download pubblici)

38. Quale protocollo di trasporto utilizza FTP?

FTP stabilisce una connessione **TCP** (orientata alla connessione) con il server FTP remoto. Nome utente e password vengono inviati come comandi FTP.

39. Perché FTP è definito un protocollo "out-of-band"? Quante connessioni TCP parallele utilizza?

È un protocollo **out-of-band** (fuori banda) perché utilizza **due connessioni TCP parallele**:

- Una per il **controllo** (persistente): invia comandi (USER, PASS, RETR, STOR, etc.) e riceve risposte
- Una per i **dati** (non persistente): trasferisce effettivamente i file (si apre e chiude per ogni trasferimento)

40. Qual è la differenza tra la connessione di controllo e la connessione dati in FTP?

- **Connessione di controllo:** persistente, rimane aperta per tutta la sessione. Utilizza tipicamente la porta 21. Trasporta comandi e risposte (testuali).
- **Connessione dati:** non persistente, viene aperta per ogni trasferimento file (o per il listing delle directory). Utilizza tipicamente la porta 20 in modalità attiva, o una porta casuale in modalità passiva.

41. Qual è la differenza tra modalità attiva e modalità passiva in FTP?

- **Modalità attiva:** la connessione di controllo è avviata dal client; la **connessione dati è avviata dal server** verso il client. Il server si connette a una porta casuale del client (specificata nel comando PORT). Può causare problemi con firewall client.
- **Modalità passiva:** **entrambe le connessioni** (controllo e dati) sono **avviate dal client**. Il server comunica al client su quale porta ascoltare (comando PASV), e il client si connette a quella porta. È preferita quando il client è dietro un firewall.

Sottosezione 8.2.2 – TFTP

42. Cosa significa l'acronimo TFTP?

Trivial File Transfer Protocol (protocollo di trasferimento file banale/triviale).

43. Per quali scopi è stato progettato TFTP?

Per scopi specifici in cui è necessario trasferire file in modo **rapido e semplice** senza richiedere tutte le funzionalità avanzate offerte da FTP. È ideale per dispositivi con risorse limitate o firmware incorporato.

44. Quale protocollo di trasporto utilizza tipicamente TFTP?

Sebbene sia progettato per l'uso con **UDP** (User Datagram Protocol), potrebbe potenzialmente essere utilizzato con altri protocolli di trasporto. L'uso di UDP lo rende più leggero ma senza le garanzie di affidabilità di TCP (l'affidabilità è implementata a livello applicativo).

45. Perché TFTP è adatto per dispositivi con risorse limitate o firmware incorporato?

Perché è progettato per essere **facile da implementare** e di **piccole dimensioni** (codice ridotto). Può essere facilmente incluso nei firmware dei dispositivi (es. router, switch, stampanti di rete, terminali boot diskless) che hanno poca memoria e poca potenza di calcolo.

46. Quali sono i cinque tipi di messaggi del protocollo TFTP?

1. **Richiesta di lettura (Read Request, RRQ)** : inviata dal client per richiedere il trasferimento di un file dal server
2. **Richiesta di scrittura (Write Request, WRQ)** : inviata dal client per inviare un file al server
3. **Dati (DATA)** : inviati dal server al client (o viceversa per scrittura), contengono blocchi di dati del file (tipicamente 512 byte)
4. **Acknowledgment (ACK)** : inviato dal client al server per confermare il ricevimento dei dati
5. **Errore (ERROR)** : inviato dal server (o dal client) al client in caso di errore durante il trasferimento

47. TFTP supporta la modalità attiva e passiva? Come funzionano?

Sì, TFTP supporta entrambe le modalità:

- **Modalità attiva:** la connessione di controllo è inizializzata dal client (il client invia RRQ o WRQ a una porta nota del server, tipicamente 69). Il server risponde da una porta casuale, e il trasferimento procede con pacchetti DATA e ACK su quella coppia di porte.

- **Modalità passiva: entrambe le connessioni** (controllo e dati) sono inizializzate dal client (meno comune, concettualmente simile a FTP passivo).

Sezione 8.3 – DNS – Domain Name System

48. Perché il vecchio sistema basato sul file hosts non poteva sostenersi con l'espansione di Internet?

Il file hosts era distribuito a tutti gli host (in un ambiente centralizzato). Con l'espansione di Internet (meno di 100 host iniziali → milioni di host), questo modello non poteva sostenersi perché:

- Mantenere un file hosts aggiornato su tutti gli host sarebbe stato **impossibile** (cambiamenti continui)
- Il file sarebbe diventato **enorme** e ingestibile
- Non c'era un meccanismo automatico per distribuire gli aggiornamenti
- Non c'era **scalabilità**

49. Cosa significa l'acronimo DNS? A cosa serve?

DNS significa **Domain Name System** (sistema dei nomi di dominio). È un sistema per introdurre nomi in formato ASCII per identificare gli host e separare così i nomi dagli indirizzi IP. Serve a tradurre i nomi di dominio (es. `www.example.com`) in indirizzi IP (es. `93.184.216.34`) e viceversa.

50. Quale RFC definisce il DNS?

DNS è definito nell'**RFC 1035**.

51. Com'è strutturato lo spazio dei nomi DNS?

È strutturato **gerarchicamente**, simile a un indirizzo postale (o a una struttura ad albero). Ha una **directory radice** (root) rappresentata da un punto (.) seguita da due gerarchie di dominio di primo livello: **organizzativa** (es. `.com`, `.org`, `.edu`, `.gov`, `.net`) e **geografica** (es. `.it`, `.fr`, `.uk`, `.de`, `.jp`).

52. Qual è il simbolo che rappresenta la directory radice del DNS?

Il punto (.). È la radice dell'albero DNS, anche se nella scrittura comune viene spesso omissa (es. `www.example.com`. con il punto finale è il nome completo, FQDN – Fully Qualified Domain Name).

53. Quali sono le due gerarchie di dominio di primo livello?

- **Dominio organizzativo (gTLD – generic Top Level Domain)** : `.com`, `.org`, `.edu`, `.gov`, `.net`, `.mil`, `.int`
- **Dominio geografico (ccTLD – country code Top Level Domain)** : `.it` (Italia), `.fr` (Francia), `.uk` (Regno Unito), `.de` (Germania), `.jp` (Giappone), etc.

54. Cos'è un resolver DNS? Fai un esempio (come `gethostbyname()`).

Un **resolver** è una procedura di libreria (tipicamente nel sistema operativo) che un'applicazione chiama per risolvere un nome di dominio in un indirizzo IP. L'applicazione richiama il resolver, che si occupa di interrogare i server DNS. L'esempio fornito è **gethostbyname()** (chiamata di sistema in C per la risoluzione DNS). Con l'indirizzo IP ottenuto, l'applicazione può stabilire una connessione TCP o inviare pacchetti UDP.

55. Descrivi i passaggi del processo di risoluzione dei nomi DNS.

1. Un server DNS è configurato con una cache iniziale (hints) contenente gli indirizzi noti dei server di nome radice
2. Quando un client effettua una **query ricorsiva** a quel server, questo elabora la richiesta usando la cache (se già possiede la risposta) o esegue i passaggi successivi per conto del client
3. Viene effettuata una query a uno dei **server radice** (root server) per trovare il server autoritativo per il dominio di primo livello richiesto (es. .com)
4. Viene ricevuta una risposta che indica il nameserver per quella risorsa (es. nameserver per example.com)
5. Il server "**attraversa l'albero**" da destra a sinistra, passando da nameserver a nameserver (es. root → .com → example.com → www.example.com), fino al passo finale che restituisce l'indirizzo IP dell'host
6. L'indirizzo IP della risorsa viene quindi restituito al client

56. Perché le query DNS usano tipicamente UDP? Quando passano a TCP?

Le query DNS usano tipicamente **UDP** (User Datagram Protocol) perché sono **leggere** e devono avvenire **rapidamente** senza l'overhead di stabilire una connessione TCP. Le query standard hanno dimensioni piccole (fino a 512 byte).

Passano a **TCP** quando:

- La risposta supera i **512 byte** (ad esempio, risposte con molti record o con DNSSEC)
- Si effettuano **trasferimenti di zona** (zone transfer) tra server DNS

57. A cosa serve il campo Identifier nel protocollo DNS?

Il campo **Identifier** è un campo ID a 16 bit che **corrisponde le richieste e le risposte**. Il client imposta un identificatore univoco nella query, e il server restituisce lo stesso identificatore nella risposta. Permette al client di abbinare ogni risposta alla query corrispondente (essenziale quando ci sono più query in sospeso concorrenti su UDP).

58. Cosa indica il flag QR (Query/Response)?

È un campo a 1 bit che designa se il pacchetto è una **richiesta (Query)** o una **risposta (Response)**. QR=0 per una query, QR=1 per una risposta.

59. Cosa indica il flag AA (Authoritative Answer)?

È un campo a 1 bit che indica una **risposta autorevole**. Il bit è impostato su 1 se il server che ha fornito la risposta è **autorevole** per il dominio in questione (cioè ha l'autorità sui dati). Se impostato a 0, è una **risposta non autorevole** (proveniente da una cache).

60. Cosa indica il flag TC (Truncation)?

Campo a 1 bit per la **troncatura** (truncation). Indica che la risposta è stata **troncata** perché era più lunga di 512 byte ed è stata inviata tramite UDP. Di solito, il client deve **riprovare con TCP** (che non ha limiti di lunghezza) per ottenere la risposta completa.

61. Cosa significa il flag RD (Recursion Desired)?

Recursion Desired (ricorsione desiderata) è un campo a 1 bit. Indica che il client sta chiedendo al server di **attraversare l'albero** (eseguire la risoluzione completa) per conto del client e di restituire semplicemente la risposta, anziché indicare dove cercare successivamente (risposta iterativa).

62. Cosa significa il flag RA (Recursion Available)?

Recursion Available (ricorsione disponibile) è un campo a 1 bit in cui un server DNS indica a un client se **supporta o meno la ricorsione**. Se RA=1, il server è disposto a fare query ricorsive; se RA=0, il client deve fare query iterative.

63. Quali sono i possibili valori di RCode (Response Code) e cosa indicano?

RCode è un campo a 4 bit. I valori significativi sono:

- **0**: nessun errore (No error)
- **1**: errore di formato (Format error)
- **2**: errore del server (Server failure)
- **3**: errore di nome (Name error) – il dominio non esiste (NXDOMAIN)
- **4**: non implementato (Not implemented)
- **5**: rifiutato (Refused)
- **6**: il nome esiste ma non dovrebbe
- **7**: esiste un record di risorsa che non dovrebbe
- **8**: un record di risorsa che dovrebbe esistere non esiste
- **9**: la risposta non è autorevole
- **10**: il nome nella risposta non è all'interno della zona specificata

64. Cosa sono QDCount, ANCount, NSCount e ARCount?

Sono campi che indicano il **numero di record** in ciascuna sezione del messaggio DNS:

- **QDCount (Question count)** : quante domande nella sezione delle domande
- **ANCount (Answer count)** : quante risposte nella sezione delle risposte
- **NSCount (Authority count)** : quanti record di risorse nella sezione di autorità (nameserver autoritativi)
- **ARCount (Additional count)** : quanti record di risorse nella sezione aggiuntiva (es. indirizzi A/AAAA dei nameserver)

65. Quali sono gli elementi che compongono una query DNS (qname, qtype, qclass)?

- **qname**: il nome di dominio da risolvere (es. www.ripe.net)
- **qtype**: il tipo di query (es. A, AAAA, MX, CNAME, PTR, SRV, TXT, NS)
- **qclass**: la classe della query (generalmente **IN** per Internet; CH e HS sono poco utilizzati oggi)

66. Cosa sono i Resource Records (RR)? Quali informazioni contengono?

I **Resource Records** (record di risorse) sono associati a ogni dominio e contengono informazioni specifiche. Ogni record contiene:

- **Name:** il nome di dominio a cui il record appartiene
- **TTL** (Time To Live): tempo di vita (quanto tempo il record può essere memorizzato in cache)
- **Class:** generalmente IN (Internet)
- **Type:** tipo di record (es. A, MX, NS, CNAME, etc.)
- **Value:** il valore effettivo del record (indirizzo IP, nome, etc.)

67. Spiega i seguenti tipi di record DNS: A, AAAA, MX, CNAME, PTR, NS, SOA, TXT, HINFO, SRV.

- **A** (Address): contiene l'indirizzo IPv4 di un host
- **AAAA:** contiene l'indirizzo IPv6 di un host
- **MX** (Mail eXchange): specifica il nome dell'host configurato per accettare la posta per quel dominio (con priorità)
- **CNAME** (Canonical Name): permette di creare alias (un nome alternativo per un host)
- **PTR** (Pointer): utilizzato per le ricerche inverse (da indirizzo IP a nome)
- **NS** (Name Server): specifica il server di nomi autoritativo per un dominio
- **SOA** (Start of Authority): fornisce le informazioni principali sul server di nomi della zona (contatto amministrativo, seriale, refresh, retry, expire, TTL minimo)
- **TXT** (Text): testo libero, utilizzato facoltativamente per auto-identificazione del dominio, verifiche (SPF, DKIM, DMARC)
- **HINFO** (Host Information): utilizzato per conoscere il tipo di macchina (CPU) e il sistema operativo
- **SRV** (Service): specifica un servizio (porta, protocollo, priorità, peso) per un dominio

68. Cosa indica il campo TTL (Time To Live) in un record DNS?

Il **TTL** (Time To Live) indica il tempo di vita del record (in secondi). Specifica per quanto tempo il record può essere **memorizzato nella cache** da resolver e server DNS prima di essere considerato obsoleto e dover essere rinfrescato con una nuova query. TTL più alti riduono il carico sui server DNS ma possono causare tempi di propagazione più lunghi in caso di cambiamenti.

Sottosezione 8.3.2 – Server DNS

69. Come viene diviso lo spazio dei nomi DNS tra i server? Cosa sono le zone?

Lo spazio dei nomi DNS viene diviso in **zone non sovrapposte**. Una **zona** è una porzione contigua dello spazio dei nomi DNS (un sottoalbero) che è gestita da uno specifico server DNS autoritativo. Un server può essere responsabile di una o più zone. La zona contiene i record di risorse per i domini che gli sono delegati.

70. Qual è la differenza tra un server DNS autorevole e un server cache?

- **Server autorevole (authoritative server)** : contiene i record nel suo file di zona (dati originali) e risponde solo alle query per i dati sotto la sua autorità. Se non può rispondere, indica l'autorità (delega) ma non effettua query in modo ricorsivo.
- **Server cache (caching resolver)** : non ha nomi autoritativi propri; esegue solo query ricorsive e caching. Salva le risposte per future richieste entro un certo periodo di tempo (TTL). Serve ad accelerare le query DNS per la rete.

71. Cosa sono le query ricorsive in DNS?

Le **query ricorsive** sono quelle in cui il client chiede al server di **svolgere tutto il lavoro per lui**. Il client invia nella sua query il flag **RD (RECURSION DESIRED)**. Il server DNS (se supporta la ricorsione e se il flag RA è 1) si assume l'onere di contattare tutti i server DNS necessari (root, TLD, authoritative) fino a ottenere la risposta definitiva, che restituisce al client. Il client riceve direttamente la risposta, senza dover fare altre query.

72. Cosa sono le query iterative in IPv6? (in realtà "iterative" in DNS)

Le **query iterative** sono l'opposto delle query ricorsive. Quando vengono utilizzate, il server **non cerca la risposta per il client** (a meno che non sia la prima domanda e risposta), ma piuttosto **indica al client dove guardare successivamente**. Quindi, se il client chiede per chat.google.com, il server gli indica di controllare con i server .com e considera il suo lavoro completato. Il client deve quindi ripetere la query al server successivo (iterazione).

73. Qual è la differenza tra risposte autorevoli e non autorevoli?

- **Risposte autorevoli**: provengono direttamente da un server dei nomi che ha **autorità** sul record in questione (cioè contiene i dati originali nel suo file di zona). Il flag AA è impostato a 1.
- **Risposte non autorevoli**: provengono da un'altra fonte, ad esempio da un server **cache** che ha memorizzato la risposta da una query precedente, non dalla fonte originale. Il flag AA è impostato a 0.

74. Cosa sono le query inverse (reverse DNS)? A cosa servono?

Le **query inverse** semplicemente **invertono il senso** delle ricerche DNS: vanno dall'indirizzo IP al nome (invece che dal nome all'indirizzo IP). Le query in avanti (forward) sono le normali query di nome-a-IP. Le query inverse utilizzano il dominio speciale **in-addr.arpa** per IPv4 (es. 65.43.32.192.in-addr.arpa per l'IP 192.32.43.65) e **ip6.arpa** per IPv6. Servono per applicazioni come logging di sicurezza, verifica di identità, anti-spam, etc.

75. Cosa sono i trasferimenti di zona (zone transfer)? Che differenza c'è tra AXFR e IXFR?

I **trasferimenti di zona** sono il mezzo attraverso il quale i **server slave** (secondari) recuperano i record dai **server master** (primari) per scopi di backup e ridondanza.

- **AXFR (Full Zone Transfer)**: trasferimento completo dell'intera zona (tutti i record). Viene usato per la sincronizzazione iniziale o quando ci sono molti cambiamenti.
- **IXFR (Incremental Zone Transfer)**: trasferimento incrementale (solo le modifiche). Più efficiente perché trasferisce solo i record cambiati, non l'intera zona.

76. Perché i trasferimenti di zona usano TCP invece di UDP?

I trasferimenti di zona avvengono tramite **TCP** perché i dati trasferiti sono solitamente **consistenti** (grandi volumi di record) e molto probabilmente **superiori a 512 byte**. UDP ha un limite di 512 byte per le risposte DNS standard; i trasferimenti di zona superano quasi sempre questo limite. TCP permette anche un trasferimento affidabile (con ACK e ritrasmissioni) senza perdite di dati.

77. Perché i trasferimenti di zona sono sensibili dal punto di vista della sicurezza?

Sono sensibili perché quando qualcuno **sa cosa e dove si trovano le tue risorse** (la topologia della rete, i nomi e indirizzi IP dei server, le configurazioni), può **pianificare un attacco** contro di te (es. identificare obiettivi vulnerabili, mappare la rete). Per questo motivo, i trasferimenti di zona dovrebbero essere **consentiti solo da sistemi approvati** (autenticazione e restrizione degli IP dei server slave).

Sottosezione 8.3.3 – Problemi del DNS

78. Come aiuta il DNS nella consegna della posta elettronica?

Il DNS aiuta gli agenti di consegna della posta (MTA) a inviare la posta su Internet utilizzando i **record MX (Mail eXchange)**. Quando un MTA deve consegnare un'email a un dominio (es. example.com), interroga il DNS per ottenere il record MX di quel dominio, che specifica il nome dell'host (server di posta) configurato per accettare la posta per quel dominio (e la priorità, nel caso ci siano più server).

79. Quali sono i problemi di base del DNS (testo non cifrato, UDP, struttura ad albero, fiducia)?

- **Testo non cifrato:** le query e le risposte DNS sono in chiaro (senza crittografia), quindi intercettabili
- **UDP semplice, senza sessioni:** le query sono su UDP senza connessione, vulnerabili a spoofing e iniezione di risposte false
- **Struttura ad albero con deleghe:** dipendenza da una catena di fiducia; se un anello è compromesso, l'intero albero sotto di esso può essere compromesso
- **Ogni entità è responsabile di una parte limitata di esso:** la responsabilità è distribuita, ma anche la vulnerabilità è distribuita
- **I resolver sono vittime di attacchi, dirottamenti e errori:** i resolver possono essere ingannati da risposte false (cache poisoning) o configurati male
- **È necessaria fiducia:** bisogna fidarsi dei server DNS upstream e delle deleghe

80. Cosa significa l'acronimo DNSSEC? Quale problema risolve?

DNSSEC significa **DNS Security Extensions** (estensioni di sicurezza DNS). Aggiunge strati al DNS per renderlo **verificabile**, garantendo **autenticità e integrità** dei dati. Risolve il problema della mancanza di sicurezza nel DNS tradizionale, prevenendo attacchi come:

- **Cache poisoning** (avvelenamento della cache)
- **Spoofing** (falsificazione) di risposte DNS
- Iniezione di record falsi

81. Come funziona DNSSEC? Cosa sono le firme RRSIG e i record DS?

DNSSEC funziona firmando i **Resource Record Sets** (insiemi di record di risorse) con la **chiave privata DNSKEY**. Il processo:

1. Il proprietario della zona genera una coppia di chiavi (pubblica/privata)
2. I record della zona vengono firmati con la chiave privata, generando **firme RRSIG** (Resource Record Signature)
3. Per verificare le firme RRSIG, il resolver deve disporre delle **chiavi pubbliche DNSKEY**.
4. Il **genitore** garantisce l'autenticità della chiave del figlio firmando l'hash di essa (record **DS – Delegation Signer**)

5. Questo processo viene ripetuto per il genitore e il nonno, creando una **catena di fiducia** dalla radice verso il basso
6. Nell'ideale, viene distribuita una sola chiave pubblica DNSKEY (quella della radice)

82. Quali sono i principali software DNS menzionati (BIND, Unbound, NSD, Microsoft DNS, Knot DNS, PowerDNS)?

- **BIND** (Berkeley Internet Name Domain): il software DNS open source più ampiamente utilizzato su Internet. Mantenuto da ISC. Versione attuale BIND 9.11.4-P2 (all'epoca del testo).
- **Unbound**: un resolver DNS di caching (non fa da server autorevole)
- **NSD** (Name Server Daemon): un nameserver **solo autorevole** (non fa caching ricorsivo)
- **Microsoft DNS**: fornito con Windows Server
- **Knot DNS**: un nameserver solo autorevole
- **PowerDNS**: supporta diversi backend di archiviazione dei dati (file, database SQL, etc.)

83. Cosa è BIND? Chi lo mantiene?

BIND (Berkeley Internet Name Domain) è il software DNS open source più ampiamente utilizzato su Internet. È mantenuto dall'**Internet Systems Consortium (ISC)**. È sia un server DNS autorevole che un resolver ricorsivo/cache.

Sezione 8.4 – World Wide Web

84. Cos'è il World Wide Web dal punto di vista dell'utente?

Dal punto di vista degli utenti, il Web è una **vasta raccolta mondiale di documenti o pagine Web** (spesso definite per brevità solamente pagine). Ogni pagina può contenere **collegamenti (link)** ad altre pagine situate ovunque nel mondo. Gli utenti possono seguire un collegamento facendo click su di esso e raggiungendo così la pagina indicata; il processo può essere ripetuto indefinitamente.

85. Cosa sono i link (collegamenti) ipertestuali?

Sono elementi (tipicamente testo sottolineato o immagini) che puntano ad altre pagine Web. L'idea di fare in modo che ogni pagina puntasse a un'altra è l'**ipertesto (hypertext)**. Quando l'utente fa click su un link, il browser carica e visualizza la pagina collegata.

86. Cos'è un browser? Fai alcuni esempi.

Un **browser** è un programma che visualizza le pagine Web e rileva i click del mouse sugli elementi in esse visualizzati. Quando viene selezionato un elemento, il browser segue il collegamento ipertestuale e preleva la pagina selezionata. Esempi: **Firefox, Internet Explorer, Chrome** (i più popolari al momento della scrittura del testo).

87. Qual è la differenza tra pagina statica e pagina dinamica?

- **Pagina statica**: è sempre uguale (il documento viene semplicemente letto da un disco). Il contenuto non cambia a meno che il file non venga modificato manualmente.
- **Pagina dinamica**: viene **generata su richiesta** da un programma (es. script lato server) o contiene del codice eseguibile. Si presenta in modo diverso ogni volta che viene visualizzata (es. risultati di

ricerca, homepage personalizzate con raccomandazioni, carrello della spesa).

88. Cosa sono i cookie e a cosa servono?

I **cookie** sono un meccanismo che permette ai siti Web di **tenere traccia delle informazioni sugli utenti** tra una visita e l'altra (o tra pagine diverse dello stesso sito). Servono a:

- Mantenere lo stato della sessione (es. carrello della spesa, login)
- Memorizzare preferenze dell'utente
- Tracciare il comportamento per pubblicità mirata (cookie di terze parti) Il nome deriva da un antico gergo dei programmatori ("cookie" = qualcosa che un programma passa a un altro e che potrebbe in seguito dover presentare per svolgere un lavoro).

Sottosezione 8.4.1 – Il Lato Client

89. Quali tre domande devono essere risolte prima di poter visualizzare una pagina selezionata?

1. Come si chiama la pagina?
2. Dov'è situata la pagina?
3. Com'è possibile accedere alla pagina?

La soluzione è l'**URL (Uniform Resource Locator)** che risolve tutte e tre le domande contemporaneamente.

90. Cosa significa l'acronimo URL? Da quali tre parti è composto?

URL significa **Uniform Resource Locator**. È composto da tre parti:

1. **Protocollo (o schema)** : specifica come accedere alla risorsa (es. http, https, ftp)
2. **Nome DNS della macchina** su cui è situata la pagina (es. www.cs.washington.edu)
3. **Percorso (path)** : indica in modo univoco la pagina specifica (un file da leggere o un programma da eseguire) sul server

91. Analizza l'URL: http://www.cs.washington.edu/index.html

- **Protocollo**: http
- **Nome DNS dell'host**: www.cs.washington.edu
- **Percorso**: /index.html

92. Descrivi i passaggi che il browser esegue quando un utente fa click su un collegamento ipertestuale.

1. Il browser determina l'URL (osservando che cosa è stato selezionato)
2. Il browser richiede al DNS l'indirizzo IP del server (es. www.cs.washington.edu)
3. Il DNS risponde con l'indirizzo IP (es. 128.208.3.88)
4. Il browser esegue una connessione TCP alla porta 80 su quell'indirizzo IP
5. Invia una richiesta HTTP per la pagina specifica (es. /index.html)
6. Il server invia la pagina come risposta HTTP
7. Se la pagina include URL necessari alla visualizzazione (immagini, script, video), il browser prende gli altri URL usando lo stesso procedimento

8. Il browser visualizza la pagina completa
9. La connessione TCP viene rilasciata se non vi sono altre richieste agli stessi server per un breve periodo

93. Su quale porta TCP lavora tipicamente un server HTTP?

La porta **80** (porta di default per HTTP). Per HTTPS, la porta è 443.

94. Quali sono alcuni schemi di URL comuni oltre a http (https, ftp, file, mailto, rtsp, sip, about)?

- **http**: ipertesto (HTML) – http://www.example.com/
- **https**: ipertesto con sicurezza (SSL/TLS) – https://www.bank.com/accounts/
- **ftp**: trasferimento file con FTP – ftp://ftp.cs.vu.nl/pub/minix/README
- **file**: file locale – file:///usr/suzanne/prog.c
- **mailto**: spedire e-mail – mailto:johnUser@acm.org
- **rtsp**: streaming multimediale – rtsp://youtube.com/montypython.mpg
- **sip**: chiamate multimediali – sip:vee@adversary.com
- **about**: informazioni sul browser – about:plugins

Sottosezione 8.4.1.1 – Minacce alla Sicurezza

95. Quali sono le principali minacce alla sicurezza lato server (SQL Injection, XSS, CSRF, File Inclusion)?

- **SQL Injection (iniezione SQL)** : iniezione di codice SQL malevolo nei moduli di input dell'utente, per violare, cancellare o modificare il database.
- **Cross-Site Scripting (XSS)** : iniezione di codice JavaScript dannoso in una pagina web, che viene eseguito nel browser di altri utenti (rubando cookie o eseguendo azioni dannose).
- **Cross-Site Request Forgery (CSRF)** : indurre un utente autenticato a eseguire azioni non volute su un'applicazione web (sfruttando la fiducia del sito verso l'utente).
- **File Inclusion**: inserimento di file dannosi nel server sfruttando vulnerabilità nei percorsi di inclusione dei file, portando all'esecuzione di codice arbitrario.

96. Cos'è l'SQL Injection? Come può essere prevenuta?

L'**SQL Injection** è un attacco in cui un malintenzionato sfrutta vulnerabilità nei moduli di input dell'utente (es. form di login, caselle di ricerca) per **iniettare codice SQL malevolo** nei comandi che il server esegue su un database. Esempio: inserendo ' OR '1'='1 in un campo password, l'attaccante potrebbe bypassare l'autenticazione. Può essere prevenuta con **validazione e sanitizzazione degli input** (escape dei caratteri speciali, uso di prepared statements/parametri, white-list validation).

97. Cos'è il Cross-Site Scripting (XSS)? Quali danni può causare?

Il **Cross-Site Scripting (XSS)** è un attacco in cui un attaccante riesce a **iniettare codice JavaScript dannoso** in una pagina web visualizzata da altri utenti. Questo codice (eseguendosi nel browser della vittima) può:

- Rubare informazioni sensibili (cookie di sessione, token, dati di autenticazione)
- Eseguire azioni a nome dell'utente (es. modificare impostazioni, inviare messaggi)

- Deface (modificare l'aspetto) della pagina
- Redirigere l'utente verso siti malevoli

98. Cos'è il Cross-Site Request Forgery (CSRF)? Come funziona?

Il **CSRF** (Cross-Site Request Forgery) è un attacco che consente a un attaccante di **indurre un utente autenticato** a eseguire **azioni non volute** su un'applicazione web in cui è autenticato, sfruttando la fiducia del sito verso l'utente. Esempio: l'utente è loggato su un sito bancario. L'attaccante invia un link (via email) che, quando cliccato, esegue un trasferimento di denaro. Il sito bancario, ricevendo la richiesta con i cookie di sessione validi, la esegue credendo che sia voluta dall'utente.

99. Quali sono le principali minacce alla sicurezza lato client (XSS, Clickjacking, MitM, Phishing)?

- **Cross-Site Scripting (XSS)** : codice malevolo eseguito nel contesto del browser dell'utente
- **Clickjacking**: sovrapposizione di elementi invisibili/trasparenti per indurre l'utente a cliccare su pulsanti nascosti o eseguire azioni non volute
- **Man-in-the-Middle (MitM)** : intercettazione e modifica della comunicazione tra client e server (rubare credenziali, inserire dati dannosi)
- **Phishing**: pagine web fraudolente che imitano siti legittimi per indurre l'utente a fornire informazioni personali (credenziali, dati di carte di credito)

100. Cos'è il Clickjacking? Come si può prevenire con l'intestazione X-Frame-Options?

Il **Clickjacking** è un attacco in cui gli attaccanti **sovrappongono una pagina web con contenuti invisibili o trasparenti** per indurre gli utenti a cliccare su elementi diversi da quelli che intendono (come pulsanti nascosti che eseguono azioni non volute). L'utente pensa di cliccare su un pulsante visibile, ma in realtà sta cliccando su un elemento invisibile sovrapposto (es. un "Mi piace" su Facebook, o un pulsante di trasferimento fondi).

Prevenzione: usare l'intestazione HTTP **X-Frame-Options** per impedire l'incorporazione delle pagine in iframe di terze parti. Valori tipici: **DENY** (nessuna incorporazione), **SAMEORIGIN** (solo dalla stessa origine).

101. Cos'è un attacco Man-in-the-Middle (MitM)? Come lo previene HTTPS?

Un attacco **Man-in-the-Middle (MitM)** avviene quando un attaccante si insedia **tra il client e il server**, intercettando e potenzialmente modificando la comunicazione. L'attaccante può rubare informazioni sensibili (password, numeri di carta di credito) o inserire dati dannosi.

HTTPS previene MitM crittografando la comunicazione con TLS/SSL. Anche se un attaccante intercetta il traffico, non può decifrarlo senza la chiave di sessione. Inoltre, HTTPS include meccanismi di autenticazione del server (certificati digitali), che permettono al client di verificare di stare comunicando con il server legittimo.

102. Cos'è il Phishing?

Il **Phishing** è una minaccia costante in cui gli utenti possono essere ingannati da pagine web fraudolente che **imitano siti legittimi** (es. siti bancari, servizi di posta, social network). L'obiettivo è indurre l'utente a **fornire informazioni personali**, come credenziali di accesso o dettagli di carte di credito. Spesso il phishing arriva via email con link che portano a siti clone.

103. Quali tecnologie di difesa sono menzionate (validazione input, HTTPS, CSP, cookie SameSite)?

- **Validazione e sanitizzazione degli input:** per prevenire SQL injection e XSS
- **Autenticazione e autorizzazione robuste:** per garantire che solo utenti autorizzati accedano alle risorse
- **HTTPS:** per crittografare i dati in transito e prevenire MitM
- **Content Security Policy (CSP) :** specifica quali script possono essere eseguiti sul sito web (prevenire XSS)
- **Cookie SameSite:** limitano le richieste cross-site, riducendo il rischio di CSRF
- **X-Frame-Options:** previene il clickjacking

Sottosezione 8.4.2 – Il Lato Server

104. Quali sono i passaggi eseguiti da un server Web nel suo ciclo principale?

In prima approssimazione, un server Web esegue questi passaggi:

1. Accettare una connessione TCP da un client (browser)
2. Ottenere il percorso della pagina (nome del file richiesto)
3. Ottenere il file (dal disco)
4. Inviare il contenuto del file al client
5. Rilasciare la connessione TCP

105. Quale problema risolve l'implementazione multithread di un server Web?

Risolve il problema di **servire una singola richiesta alla volta**. Con un server multithread, un modulo di front-end accetta tutte le richieste in ingresso e le distribuisce a (k) moduli di elaborazione (thread). I (k+1) thread appartengono allo stesso processo, condividendo la cache e lo spazio di indirizzamento. In questo modo, il server può gestire **molte richieste contemporaneamente** (concorrenza).

106. Quali sono i passaggi che un modulo di elaborazione di un server Web moderno può eseguire?

I passaggi (dopo che è stata stabilita la connessione TCP e i meccanismi di trasporto sicuri come SSL/TLS):

1. Risolvere il nome della pagina Web richiesta
2. Eseguire il controllo di accesso sulla pagina Web
3. Controllare la cache (se la pagina è già in cache)
4. Prelevare la pagina richiesta dal disco o eseguire un programma che la costruisca (pagina dinamica)
5. Determinare il resto della risposta (per esempio, il tipo MIME da includere)
6. Restituire la risposta al client
7. Aggiungere un elemento al registro delle operazioni svolte dal server (logging)

Sottosezione 8.4.3 – Cookie

107. Cosa sono i cookie in ambito Web? Da dove deriva il nome?

I **cookie** sono piccoli dati che il server invia al browser, che il browser memorizza e rispedisce al server nelle richieste successive. Servono a mantenere lo stato tra diverse richieste HTTP (che di per sé è stateless). Il nome deriva da un antico gergo dei programmatori: un programma chiama una procedura e ottiene qualcosa che potrebbe in seguito dover presentare per svolgere un lavoro ("cookie" come "ciò che viene passato").

108. Quali informazioni contiene tipicamente un cookie (dominio, percorso, contenuto, scadenza, sicuro)?

- **Dominio** (Domain): il dominio per cui il cookie è valido
- **Percorso** (Path): il percorso del sito per cui il cookie è valido
- **Contenuto** (Content/Value): i dati effettivi del cookie (es. ID di sessione, preferenze)
- **Scadenza** (Expires/Max-Age): data e ora di scadenza del cookie (se assente, cookie non persistente)
- **Sicuro** (Secure/SameSite): flag che indica se il cookie può essere inviato solo con connessioni sicure (HTTPS) o limitazioni cross-site

109. Qual è la differenza tra cookie persistente e cookie non persistente?

- **Cookie non persistente** (session cookie): il campo Expires è **assente**. Il browser scarta il cookie alla sua chiusura. Viene utilizzato per mantenere lo stato durante la sessione di navigazione (es. carrello della spesa).
- **Cookie persistente**: viene fornita una data e un'ora nel campo Expires (o Max-Age). Il cookie è **salvato su disco** e persiste anche dopo la chiusura del browser fino alla scadenza.

110. Come si può rimuovere un cookie dal disco rigido del client?

Per rimuovere un cookie dal disco rigido del client, un server deve inviarlo di nuovo, ma con un **tempo di scadenza già trascorso** (es. una data nel passato). Il browser, ricevendo un cookie già scaduto, lo eliminerà.

111. Cosa indica il campo "Secure" in un cookie?

Il campo **Secure** (sicuro) indica che il browser può restituire il cookie a un server **solo usando un sistema di trasporto sicuro** come **SSL/TLS** (HTTPS). Questa funzionalità è utilizzata per il commercio elettronico, il remote banking e altre applicazioni sicure. Un cookie con flag Secure non verrà mai inviato su una connessione HTTP non cifrata.

Sottosezione 8.4.4 – HTTP

112. Cosa significa l'acronimo HTTP? Quale RFC lo specifica?

HTTP significa **HyperText Transfer Protocol** (protocollo di trasferimento ipertestuale). È specificato nell'**RFC 2616** (per HTTP/1.1).

113. Su quale modello si basa HTTP (request-response)?

HTTP si basa sul modello di **domanda e risposta (request-response)**. Il client (browser) invia una richiesta al server, e il server restituisce una risposta. Normalmente implementato su **TCP**.

114. Quali sono le differenze tra HTTP 1.0 e HTTP 1.1?

- **HTTP 1.0:** dopo avere stabilito la connessione, veniva inviata una singola richiesta e restituita una singola risposta; poi la connessione TCP veniva **rilasciata** (connessione non persistente). Ogni oggetto (immagine, script, foglio di stile) richiedeva una nuova connessione.
- **HTTP 1.1:** supporta **connessioni persistenti**. È possibile stabilire una connessione TCP, inviare più richieste e ottenere più risposte senza dover riaprire la connessione ogni volta.

115. Cosa sono le connessioni persistenti in HTTP 1.1?

Le **connessioni persistenti** (keep-alive) permettono di riutilizzare la stessa connessione TCP per **più richieste HTTP** (e relative risposte). Questo ammortizza l'overhead di impostazione e rilascio di TCP su più richieste, migliorando le prestazioni, specialmente per pagine con molti oggetti (immagini, script, CSS).

116. Cosa significa "pipelining" delle richieste in HTTP?

Il **pipelining** (concatenamento) delle richieste significa inviare la **richiesta 2 prima dell'arrivo della risposta alla richiesta 1**. Il client non deve attendere la risposta alla richiesta precedente per inviarne una nuova. HTTP 1.1 supporta il pipelining su connessioni persistenti, migliorando ulteriormente le prestazioni.

117. Cosa sono i metodi HTTP? Perché sono stati resi generici?

I **metodi HTTP** sono operazioni (comandi) che il client può inviare al server. Oltre al classico **GET** (recuperare una pagina) e **POST** (inviare dati), esistono HEAD, PUT, DELETE, OPTIONS, TRACE, CONNECT. Sono stati resi generici (indipendenti dal contenuto) per supportare future applicazioni orientate agli oggetti (es. servizi web, API REST). Questa generalità ha permesso la nascita di protocolli come **SOAP** (Simple Object Access Protocol).

Sottosezione 8.4.5 – Caching

118. Cos'è il caching delle pagine Web? Quale vantaggio offre?

Il **caching** è la procedura di memorizzare in una cache (locale o condivisa) delle pagine e risorse Web recuperate per farne un uso successivo. Il vantaggio è che quando una pagina è memorizzata nella cache, non è necessario **ripetere il trasferimento** (risparmiando tempo e banda). HTTP ha un supporto integrato che aiuta i client a identificare quando possono riusare le pagine in modo sicuro.

119. Cos'è un GET condizionale (conditional GET)? Quando viene usato?

Un **GET condizionale** è una richiesta HTTP GET che include intestazioni condizionali come **If-Modified-Since** (e/o If-None-Match con ETag). Viene usato quando il client ha una copia nella cache, ma non sa se sia ancora valida. Il server risponde con:

- **304 Not Modified** se la copia nella cache è ancora valida (risposta breve)
- **200 OK** con la risorsa completa se la copia è obsoleta

120. Come fa il client a sapere se una copia nella cache è ancora valida?

Il client usa una **richiesta GET condizionale**. Invia al server la data di ultima modifica (If-Modified-Since) e/o l'ETag (If-None-Match) della copia in cache. Il server confronta queste informazioni con lo stato attuale della risorsa sul server:

- Se la risorsa non è stata modificata dopo quella data (o l'ETag corrisponde), risponde con **304 Not Modified**
 - Se è stata modificata, risponde con la nuova risorsa (200 OK)
-

Sezione 8.5 – Distribuzione dei Contenuti

121. Come è cambiato l'uso di Internet dalle comunicazioni ai contenuti?

Storicamente, Internet era basata unicamente sulle **comunicazioni**: accademici si collegavano a macchine remote per fare login, email per comunicare, e più recentemente voce/video su IP. Con la crescita del Web, Internet ha cominciato a riguardare più i **contenuti** della comunicazione: reperire informazioni, condividere file peer-to-peer (film, musica, programmi). Oggi la maggior parte della banda Internet è usata per trasmettere **video registrati**.

122. Cosa sono gli "elefanti" e i "topi" nel traffico di rete?

Sono metafore per descrivere la distribuzione del traffico:

- **Elefanti**: lunghi flussi di traffico (flussi molto grandi). Sono pochi, ma contano perché sono molto grandi e consumano la maggior parte della banda.
- **Topi**: brevi flussi di traffico. Sono molti numericamente, ma ciascuno piccolo.

Questa caratteristica è legata all'**auto-similarità** (self-similarity) del traffico di rete, che presenta molti vuoti brevi e lunghi su diverse scale temporali.

123. Cosa afferma la legge di Zipf in relazione alla popolarità dei contenuti? Qual è la formula?

La **legge di Zipf** afferma che, quando sono disponibili (N) oggetti (es. film), la frazione di tutte le richieste per il (k)-esimo più popolare è circa (C/k), dove (C) è una costante di normalizzazione tale che la somma dia 1. In formula: $[C = 1 / (1 + 1/2 + 1/3 + 1/4 + 1/5 + \dots + 1/N)]$

Conseguenza pratica: il film più popolare lo è **7 volte di più** del film numero 7. La popolarità dei contenuti segue una distribuzione a "lunga coda" (long tail).

Sottosezione 8.5.1 – Server Farm e Proxy Web

124. Cos'è una server farm? A cosa serve?

Una **server farm** è un **cluster di computer** che agisce come un singolo server Web. Serve a costruire grandi siti Web che lavorino bene (velocemente e con alta disponibilità) quando un singolo server non è sufficiente a gestire il carico. I server condividono un database comune e un front-end distribuisce le richieste tra di loro.

125. Come fa una server farm ad apparire come un singolo sito Web?

Ogni server deve avere una **copia del sito Web** (contenuti condivisi, tipicamente attraverso un database comune o storage condiviso). Un **front-end** (switch a livello data link o router IP) distribuisce le richieste in entrata su un insieme di server, usando tecniche come round-robin, hashing (es. basato su IP sorgente), o

politiche più complesse. Il front-end si assicura che tutti i pacchetti di una stessa connessione TCP vadano allo stesso server.

126. Cos'è il load balancing? Quale politica semplice si può usare (round-robin)?

Il **load balancing** (bilanciamento del carico) è la politica con cui il front-end associa le richieste ai server della server farm per distribuire il carico in modo uniforme. Una politica semplice è il **round-robin** (a turno): il front-end assegna ogni nuova richiesta al server successivo in ordine ciclico. Il front-end deve ricordarsi dell'associazione per ogni richiesta (connessione TCP).

127. Cosa sono i middlebox? Perché violano i principi della stratificazione dei protocolli?

I **middlebox** sono dispositivi (es. switch, router, NAT, load balancer) che prendono decisioni basate su informazioni provenienti dal **livello di trasporto o da livelli superiori** (es. guardano le porte TCP/UDP o i contenuti HTTP). Violano il principio di stratificazione dei protocolli perché ogni livello dovrebbe limitarsi a utilizzare solo la propria intestazione per il controllo, senza ispezionare il payload dei livelli superiori. Questo crea potenziali problemi di compatibilità e funzionamento nel lungo termine.

128. Cos'è un proxy Web? A cosa serve?

Un **proxy Web** è un server che agisce da intermediario tra i client (browser) e i server Web. Fornisce una **cache grande e condivisa** per un gruppo di client (es. un'azienda o un ISP). Serve a:

- Aumentare la velocità delle richieste Web (risposte dalla cache locale)
- Ridurre l'utilizzo di banda verso l'esterno (meno richieste ai server remoti)
- Filtrare o monitorare il traffico Web

129. Qual è la tipica configurazione di un'organizzazione che usa un proxy Web?

La tipica configurazione prevede **un solo proxy** per tutti gli utenti dell'organizzazione (azienda o ISP). I browser degli utenti sono configurati per inviare le richieste al proxy. Il proxy gestisce una cache condivisa e inoltra le richieste non soddisfatte dalla cache al server Web remoto (o a un upstream proxy).

130. Cosa sono upstream proxy e downstream proxy?

- **Upstream proxy** (proxy di livello superiore): il proxy verso cui un proxy (o un browser) inoltra le richieste che non può soddisfare dalla propria cache.
- **Downstream proxy** (proxy di livello inferiore): i proxy (o browser) che si rivolgono a un proxy per soddisfare le loro richieste.

Si possono creare gerarchie di proxy: browser → proxy aziendale → proxy ISP → server Web.

Sottosezione 8.5.1.1 – Server Farm (approfondimento)

131. Quali soluzioni esistono per distribuire le richieste su una server farm?

1. **DNS-based**: il DNS restituisce indirizzi IP diversi (round-robin) per distribuire le richieste
2. **Front-end switch (layer 4)**: distribuisce basandosi su indirizzi IP e porte TCP/UDP
3. **Front-end load balancer (layer 7)**: ispeziona il contenuto HTTP (es. URL, cookie) per distribuire

132. Come può il DNS essere usato per distribuire le richieste?

Il DNS può restituire **indirizzi IP diversi** (in round-robin) per lo stesso nome di dominio. Ad esempio, `www.example.com` può risolvere a 192.0.2.1, poi 192.0.2.2, poi 192.0.2.3, etc. I client (e i resolver) ricevono indirizzi diversi in tempi diversi, distribuendo il carico tra i server della farm. Questa soluzione è semplice ma non permette di bilanciare in modo fine (non tiene conto del carico effettivo dei server).

Sottosezione 8.5.1.2 – Proxy Web (approfondimento)

133. Quali benefici porta un proxy Web a un'azienda o ISP?

- **Maggiore velocità** nelle richieste Web (risposte dalla cache locale, senza attraversare la rete esterna)
- **Riduzione della banda** consumata verso l'esterno (le richieste soddisfatte dalla cache non consumano banda sull'uplink)
- **Risparmio economico** (aziende e ISP spesso pagano in base alla banda consumata)

134. Perché un singolo livello di cache è spesso sufficiente?

Un singolo livello di cache (es. un proxy aziendale) è spesso sufficiente per guadagnare la **maggior parte dei benefici** del caching, perché la cache locale può già servire una percentuale significativa delle richieste (es. risorse comuni, pagine visitate frequentemente, fogli di stile, script condivisi). Aggiungere ulteriori livelli di proxy introduce complessità aggiuntiva con benefici marginali decrescenti.

Sezione 8.6 – Content Delivery Network

135. Cosa significa l'acronimo CDN (Content Delivery Network)?

Content Delivery Network (rete per la distribuzione di contenuti). È un'infrastruttura distribuita in cui un provider installa un insieme di macchine (nodi) in diversi luoghi di Internet e le usa per **distribuire contenuti ai client** da nodi a loro vicini.

136. In cosa differisce una CDN dalla cache Web tradizionale?

- **Cache Web tradizionale**: sono i **client** (o i proxy) che cercano una copia della pagina richiesta in una cache vicina. La cache è popolata dinamicamente in base alle richieste degli utenti.
- **CDN**: è il **provider** (il proprietario del contenuto) che mette una copia della pagina in un insieme di nodi in differenti posizioni e **dirige il client** in modo che usi un nodo a lui vicino come server. Il contenuto viene distribuito proattivamente (push o pull in anticipo) ai nodi CDN.

137. Descrivi il funzionamento di una CDN con l'esempio dell'albero di distribuzione.

Il server di origine (origin server) nella CDN **distribuisce una copia del contenuto** ad altri nodi della CDN in diverse regioni geografiche (es. Sydney, Boston, Amsterdam). Questi nodi sono posizionati strategicamente vicino agli utenti finali. I client prendono quindi le pagine dal **nodo a loro più vicino** nella CDN (linea continua), non dal server di origine che potrebbe trovarsi dall'altra parte del mondo. In questo modo, la latenza è ridotta e il carico sul server di origine è ridotto.

138. Quali sono le tre ragioni per cui l'uso di proxy server fallisce nella pratica per le CDN?

1. I client in una certa parte della rete appartengono a organizzazioni diverse e usano proxy diversi. Le cache non sono normalmente condivise tra organizzazioni diverse per limitato beneficio e motivi di sicurezza.
2. Possono esserci più CDN (diversi provider), ma ogni client usa un solo proxy. Quale CDN dovrebbe usare il client come proxy?
3. I proxy Web sono **configurati dal client** (utente o amministratore). Possono essere configurati per beneficiare o meno di una distribuzione di contenuti di una CDN, e c'è poco che la CDN possa fare in questo senso.

139. Cos'è il mirroring? Cosa sono i mirror?

Il **mirroring** (specchiamento) è un approccio semplice per supportare un albero di distribuzione: il server di origine **replica i contenuti** (li copia) nei nodi CDN. I nodi CDN in diverse regioni della rete sono chiamati **mirror** (specchi, perché **riflettono** il contenuto del server di origine). Le pagine Web sul server di origine contengono **collegamenti espliciti** a mirror distinti, indicando di solito anche la loro locazione geografica.

140. Cos'è la DNS redirection (reindirizzamento DNS)? Come funziona?

È una tecnica che usa il DNS per dirigere i client verso il nodo CDN più vicino. Funziona così:

1. Il browser usa il DNS per risolvere il nome di dominio (es. `www.cdn.com`)
2. Il name server del dominio (gestito dalla CDN) **guarda l'indirizzo IP del client** che fa la richiesta (o il resolver)
3. Il name server restituisce l'indirizzo IP del nodo CDN **più vicino** a quel client (geograficamente o in termini di rete)
4. Esempio: un client a Sydney riceve IP del nodo Sydney; un client ad Amsterdam riceve IP del nodo Amsterdam

141. Quali due fattori determinano la scelta del nodo CDN "più vicino"?

1. **Distanza di rete** (percorso di rete breve e ad alta capacità). Le CDN usano una mappa di associazioni per tradurre l'indirizzo IP del client nella sua posizione in rete.
2. **Carico già gestito dal nodo CDN**. Se un nodo è sovraccarico, sarà lento; si possono associare alcuni client a nodi leggermente più lontani ma meno carichi (bilanciamento del carico).

142. Quale azienda fu la prima ad adottare la tecnica DNS per la distribuzione di contenuti? In che anno?

Akamai nel **1998**, quando il Web stava soffrendo sotto il carico della sua crescita iniziale. Akamai fu la prima grande CDN e divenne un leader industriale.

143. Quali vantaggi hanno gli ISP nell'ospitare un nodo CDN?

- **Riduce la banda di rete in uscita** (a pagamento), perché i contenuti sono serviti localmente all'interno della rete dell'ISP
- **Aumenta la velocità di risposta** per i clienti dell'ISP (che apprezzano un servizio più veloce)
- Dà un **vantaggio competitivo** rispetto a ISP che non hanno nodi CDN
- I benefici non costano all'ISP (la CDN installa e gestisce i nodi)

Il testo riassume: fornitori di contenuti, ISP e clienti si avvantaggiano tutti di questo schema, e la CDN fa ottimi affari.

144. Quali sono gli esempi di CDN ottimizzate per il video menzionati (Ustream, JET-Stream/Streamzilla)?

- **Ustream:** CDN ottimizzato per video, offre servizi per "Pro Broadcasting", "Corporate Internal Communications" (Ustream Align) e marketing per lead generation (Ustream Demand)
- **JET-Stream** (precedentemente noto come **Streamzilla**): CDN ottimizzato per video, sta guadagnando quote di mercato in Europa. Offre streaming adattivo con bitrate variabile tramite HTTP

145. Cosa sono i MultiCDN? Quali vantaggi offrono (bilanciamento del carico, CDN stacking, etc.)?

I **MultiCDN** sono una tecnologia emergente che combina più CDN (globali) in un'unica piattaforma. Vantaggi:

- **Bilanciamento del carico imposto:** distribuisce immediatamente il traffico senza dipendere da sistemi DNS di terze parti
- **Failover automatico:** se una CDN fallisce, viene immediatamente esclusa dal pool
- **CDN stacking:** il traffico viene bilanciato su un pool di CDN, creando di fatto il CDN più grande al mondo
- **Bilanciamento granulare:** ottimizzato per singoli ISP e fino agli indirizzi IP
- **Integrazione profonda:** permette di controllare le CDN come se fossero server edge propri
- **Mix ibrido:** ottimizzazione su mix di risorse (CDN proprietary + edge server propri + CDN globali)

146. Cosa significa "CDN stacking"?

Il **CDN stacking** è una tecnica MultiCDN in cui il traffico viene **bilanciato su un pool di CDN** (aggregazione di più CDN). Invece di usare una singola CDN, si sfrutta la capacità combinata di tutte le CDN nel pool, creando di fatto il "CDN più grande al mondo" (in termini di capacità distribuita aggregata).

147. Quale vantaggio offre lo streaming HTTP adattivo con bitrate variabile?

Lo streaming HTTP adattivo (Adaptive Bitrate Streaming, ABR) permette di **adattare dinamicamente la qualità del video** (bitrate) alle condizioni della rete. Se la larghezza di banda diminuisce, lo streaming passa a un bitrate più basso (evitando buffering); se la banda aumenta, la qualità migliora. JET-Stream (ex Streamzilla) ha implementato questa tecnologia, superando le offerte dei CDN tradizionali e aprendo nuove opportunità.

Domande trasversali e di confronto

148. Confronta IMAP e POP3 in termini di gestione dei messaggi sul server e supporto multi-dispositivo.

Caratteristica	POP3	IMAP
Gestione messaggi	Scarica i messaggi sul client (tipicamente cancella dal server)	Mantiene i messaggi sul server

Caratteristica	POP3	IMAP
Accesso multi-dispositivo	Difficile (lo stato non è sincronizzato; ciò che si fa su un dispositivo non si vede sugli altri)	Completo (lo stato è sincronizzato; letto, cancellato, spostato si riflettono su tutti i dispositivi)
Struttura cartelle	Non supportata (solo inbox locale)	Supporto completo di cartelle sul server
Accesso parziale ai messaggi	No (deve scaricare tutto)	Sì (può scaricare solo header, poi selettivamente il corpo)
Perdita dati	Rischio perdita se il client si guasta dopo il download	Nessun rischio (i messaggi sono sempre sul server)

149. Confronta FTP e TFTP in termini di: affidabilità, autenticazione, protocollo di trasporto, complessità.

Caratteristica	FTP	TFTP
Affidabilità	Alta (TCP, connessione affidabile)	Affidabilità implementata a livello applicativo su UDP
Autenticazione	Sì (username/password, accesso anonimo)	No (nessuna autenticazione)
Protocollo di trasporto	TCP	UDP (tipicamente)
Connessioni	2 connessioni (controllo persistente + dati non persistente)	1 connessione (o coppia di porte dinamiche)
Complessità	Alto (comandi multipli, due modalità, gestione directory)	Basso (solo 5 tipi di messaggi, implementazione semplice)
Dimensione	Codice grande (client e server complessi)	Piccolo (adatto per firmware embedded)
Utilizzo tipico	Trasferimento file generico, backup	Boot diskless, trasferimento configurazioni su router/switch

150. Confronta HTTP 1.0 e HTTP 1.1 in termini di connessioni e pipeline.

Caratteristica	HTTP 1.0	HTTP 1.1
Connessioni	Non persistenti (una connessione per richiesta)	Persistenti (riutilizzo connessione per più richieste)
Pipeline	Non supportato	Supportato (inviare richiesta 2 prima della risposta 1)
Overhead	Alto (TCP setup/release per ogni risorsa)	Basso (un setup per più risorse)

Caratteristica	HTTP 1.0	HTTP 1.1
Prestazioni	Scadenti per pagine con molti oggetti	Buone per pagine con molti oggetti

151. Confronta server farm, proxy Web e CDN come soluzioni per migliorare le prestazioni del Web.

Soluzione	Scopo	Come funziona	Limitazioni
Server farm	Aumentare capacità di elaborazione (lato server)	Cluster di server + front-end bilanciamento carico	Non riduce latenza geografica
Proxy Web	Cache condivisa (lato client/ISP)	Cache locale per gruppo di utenti; serve le risposte senza contattare il server remoto	La cache è popolata dinamicamente; non serve il primo utente
CDN	Distribuzione geografica del contenuto	Contenuto replicato proattivamente su nodi vicini agli utenti; reindirizzamento DNS	Più complessa; richiede coordinamento con ISP

152. Spiega la differenza tra una query DNS ricorsiva e una iterativa.

- **Ricorsiva:** il client chiede al server DNS di **svolgere tutto il lavoro** (seguire tutte le deleghe) e restituire la risposta definitiva. Il server contatta i root server, i TLD server, i nameserver autoritativi, poi restituisce la risposta al client.
- **Iterativa:** il server DNS **non** cerca la risposta per il client (a meno che non ce l'abbia già in cache). Invece, **indica al client dove guardare successivamente** (es. "chiedi al server .com"). Il client deve ripetere la query al server successivo (iterazione).

153. Confronta i record DNS A, AAAA e MX: a cosa serve ciascuno?

Record	Scopo	Esempio
A	Mappa un nome di dominio a un indirizzo IPv4	<code>example.com. 3600 IN A 93.184.216.34</code>
AAAA	Mappa un nome di dominio a un indirizzo IPv6	<code>example.com. 3600 IN AAAA 2606:2800:220:1:248:1893:25c8:1946</code>
MX	Specifica il server di posta per un dominio (con priorità)	<code>example.com. 3600 IN MX 10 mail.example.com.</code>

154. Qual è la differenza tra un server DNS autorevole e un server DNS cache?

- **Server autorevole:** contiene i record originali (file di zona) e **risponde in modo autoritativo** (flag AA=1). Non esegue query ricorsive per conto dei client (salvo configurazione speciale).
- **Server cache** (caching resolver): non ha dati autoritativi propri. **Esegue query ricorsive** per conto dei client e **memorizza le risposte** (cache) per un tempo pari al TTL. Riduce la latenza e il carico sui server autorevoli.

155. Confronta le minacce alla sicurezza lato server e lato client nel Web.

Minaccia	Lato server	Lato client	Descrizione
SQL Injection	✓	–	Iniezione di codice SQL in input, compromette database
XSS	✓	✓	Iniezione di JavaScript; lato server (se non sanitizza), lato client (esecuzione)
CSRF	✓	–	Sfrutta la sessione autenticata dell'utente verso il server
File Inclusion	✓	–	Inclusione di file dannosi sul server
Clickjacking	–	✓	Interfaccia ingannevole con elementi trasparenti
Man-in-the-Middle	–	✓	Intercettazione comunicazione tra client e server
Phishing	–	✓	Pagina fraudolenta che imita un sito legittimo

156. Confronta i cookie persistenti e non persistenti: quando viene cancellato ciascuno?

- **Cookie non persistente** (session cookie): viene cancellato **alla chiusura del browser**. Non ha data di scadenza. Usato per mantenere lo stato durante la sessione (carrello, login temporaneo).
- **Cookie persistente**: viene cancellato alla **data di scadenza** specificata nel campo **Expires** (o dopo il tempo nel campo **Max-Age**). Sopravvive alla chiusura del browser e viene salvato su disco.

157. Qual è la differenza architetturale tra una CDN e una rete P2P per la distribuzione di contenuti?

- **CDN** (Content Delivery Network): architettura **centralizzata/distribuita controllata**. Un provider commerciale installa nodi (server) in posizioni strategiche della rete. Il contenuto è distribuito da server professionali. Esempi: Akamai, Cloudflare, Amazon CloudFront.
- **P2P** (Peer-to-Peer): architettura **decentralizzata**. I computer degli utenti (peer) **usano le proprie risorse** (banda in uplink, storage) per distribuire contenuti tra di loro, **senza server separati o punti centrali di controllo**. Ogni peer può sia scaricare che caricare. Esempi storici: BitTorrent, eMule.